

by importing `Union` from the built-in `typing` module. Specify a range of types inside square brackets following the `Union` class name:

---

```
from typing import Union
spam: Union[int, str, float] = 42
spam = 'hello'
spam = 3.14
```

---

In this example, the `Union[int, str, float]` type hint specifies that you can set `spam` to an integer, string, or floating-point number. Note that it's preferable to use the `from typing import x` form of the `import` statement rather than the `import typing` form and then consistently use the verbose `typing.x` for type hints throughout your program.

You might specify multiple data types in situations where a variable or return value could have the `None` value in addition to another type. To include `NoneType`, which is the type of the `None` value, in the type hint, place `None` inside the square brackets rather than `NoneType`. (Technically, `NoneType` isn't a built-in identifier the way `int` or `str` is.)

Better yet, instead of using, say, `Union[str, None]`, you can import `Optional` from the `typing` module and use `Optional[str]`. This type hint means that the function or method could return `None` rather than a value of the expected type. Here's an example:

---

```
from typing import Optional
lastName: Optional[str] = None
lastName = 'Sweigart'
```

---

In this example, you could set the `lastName` variable to `None` or a `str` value. But it's best to make sparing use of `Union` and `Optional`. The fewer types your variables and functions allow, the simpler your code will be, and simple code is less bug prone than complicated code. Remember the Zen of Python maxim that simple is better than complex. For functions that return `None` to indicate an error, consider raising an exception instead. See “Raising Exceptions vs. Returning Error Codes” on page 178.

You can use the `Any` type hint (also from the `typing` module) to specify that a variable, parameter, or return value can be of any data type: